

NUCLEO-G071RB IoT Prototype – Complete Guide

Overview

This document walks you through building a *passive* IoT node on the **ST NUCLEO-G071RB** that

- reads **BME680** environmental data over **I²C**,
- controls an **SX1262 LoRa** radio over **SPI**,
- accepts ASCII commands from a PC over **UART** (USB virtual COM), and
- is developed entirely in **STM32CubeIDE** (no RTOS) with terminal interaction via **PuTTY**.

The goal is for you to be able to power-up, flash, open PuTTY, and immediately issue commands such as `READ` and `SEND,Hello`.

Bill of Materials & Wiring

Qty	Part	Notes
1	NUCLEO-G071RB	On-board ST-Link/V2-1 (USB Micro-B)
1	BME680 breakout (3.3 V)	I ² C interface, addr 0×76/0×77
1	SX1262 LoRa HAT/module	3.3 V logic (e.g. EBYTE E22-900M22S)
≈ 12	Dupont M-F wires	150 mm recommended
1	USB-A ↔ Micro-B cable	Power / SWD
1	868 / 915 MHz antenna	Match your band

Pin Map

Function	Module Pin	NUCLEO-G071RB Pin	Notes
BME680 VCC	3V3	3V3	--
BME680 GND	GND	GND	--
BME680 SCL	SCL	PB6 (I2C1_SCL)	Pull-ups on breakout
BME680 SDA	SDA	PB7 (I2C1_SDA)	"
SX1262 VCC	3V3	3V3	250 mA peak
SX1262 GND	GND	GND	--
SX1262 SCK	SCK	PA5 (SPI1_SCK)	--

Function	Module Pin	NUCLEO-G071RB Pin	Notes
SX1262 MISO	MISO	PA6 (SPI1_MISO)	--
SX1262 MOSI	MOSI	PA7 (SPI1_MOSI)	--
SX1262 NSS	NSS	PB0 (GPIO OUT)	Chip-select
SX1262 RESET	RESET	PA0 (GPIO OUT)	Active low
SX1262 BUSY	BUSY	PA1 (GPIO IN)	Poll pin
SX1262 DIO1	DIO1	PC4 (EXTI)	Packet-done IRQ
UART TX	PA2	USB VCP to PC	115200 bps
UART RX	PA3	USB VCP to PC	--

Tip: Route NSS/BUSY/RESET/DIO1 to free male headers close to the Arduino connector so the HAT can drop in.

1 Create the Project (STM32CubeIDE $\geq$ 1.15)

1. **File** ▶ **New** ▶ **STM32 Project**. Choose **NUCLEO-G071RB** from the *Board Selector*.
2. Name it **IoT_LoRa_BME680** and click *Finish* to open the `.ioc` pin-config tool.

1.1 Clock

- In **Clock Configuration** tab, select **HSI16** → **PLL** → **64 MHz** (set PLLM=1, PLLN=8).
- Verify APB1/APB2 running at 64 MHz; SysTick 1 ms.

1.2 Peripherals (Pinout & Configuration tab)

Peripheral	Mode	Pins	Key Settings
I2C1	I ² C	PB6 SCL, PB7 SDA	Timing 0x2000090E (400 kHz)
SPI1	Full-Duplex Master	PA5 SCK, PA6 MISO, PA7 MOSI	Prescaler ÷8 (8 MHz); <i>Disable</i> hardware NSS
USART2	Asynchronous	PA2 TX, PA3 RX	115200 8N1, no parity
GPIO	Outputs	PB0 <code>SX1262_NSS</code> , PA0 <code>SX1262_RST</code>	Initial HIGH
	Inputs	PA1 <code>SX1262_BUSY</code>	Floating
	EXTI	PC4 <code>SX1262_DIO1</code>	Rising edge IRQ
NVIC	–	EXTI line 4	Enable IRQ

Hit **Project** ▶ **Generate Code**.

2 Pull in Third-Party Drivers

Create two folders *Core/Drivers/bme680* and *Core/Drivers/sx126x*.

2.1 BME680 (Bosch Sensortec)

1. Download the official C driver (v3.5.9+).
2. Copy `bme680.{c,h}` into *bme680/*.
3. In `bme680.h` set

```
#define BME680_FLOAT_POINT_COMPENSATION 1
```

2.2 SX1262 (Semtech)

1. From the Semtech SX126x driver bundle copy:
2. `sx126x.c/h`
3. `sx126x_regs.h`, `sx126x_defs.h`
4. Provide a minimal hardware-abstraction layer `sx126x_hal.c/h` (see §4.3).
5. Add include paths: *Project* ▶ *Properties* ▶ *C/C++ Build* ▶ *MCU GCC Compiler* ▶ *Includes* → `Core/Drivers/bme680`, `Core/Drivers/sx126x`.

3 Application Code Skeleton

Below are the key user sections you need to paste into CubeMX-generated files. Full sources are in *Appendix D*.

3.1 `main.c` (excerpt)

```
/* USER CODE BEGIN Includes */
#include "bme680.h"
#include "sx126x.h"
#include "console.h"
/* USER CODE END Includes */

static struct bme680_dev bme;
static sx126x_t radio;

static void Sensors_Init(void)
{
    bme.intf = BME680_I2C_INTF;
```

```

    bme.write = platform_i2c_write;
    bme.read = platform_i2c_read;
    bme.delay_ms = HAL_Delay;
    bme.dev_id = (BME680_I2C_ADDR_SECONDARY << 1); // 0x77<<1 = 0xEE

    if (bme680_init(&bme) != BME680_OK) Error_Handler();

    bme.tph_sett.os_hum = BME680_OS_2X;
    bme.tph_sett.os_pres = BME680_OS_4X;
    bme.tph_sett.os_temp = BME680_OS_8X;
    bme.tph_sett.filter = BME680_FILTER_SIZE_3;
    bme.power_mode = BME680_FORCED_MODE;
}

static void LoRa_Init(void)
{
    sx126x_hal_init(&radio, &hspi1,
                   SX126X_FREQ_868_100_MHZ, SX126X_TX_DBM_14,
                   SX126X_SPREADING_FACTOR_7, SX126X_BW_125_KHZ,
                   SX126X_CR_4_5);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_I2C1_Init();
    MX_SPI1_Init();
    MX_USART2_UART_Init();

    console_init(&huart2);
    Sensors_Init();
    LoRa_Init();

    console_puts("\r\nNUCLEO-G071RB IoT ready>\r\n");

    uint32_t t0 = HAL_GetTick();
    while (1)
    {
        console_process_rx(); // Handle UART
        if (HAL_GetTick() - t0 >= 1000) // 1 s interval
        {
            t0 += 1000;
            bme680_set_sensor_mode(&bme);
            HAL_Delay(bme.get_meas_dur(&bme) + 5);
        }
    }
}

```

```

        struct bme680_field_data d;
        if (bme680_get_sensor_data(&d, &bme) == BME680_OK)
            console_printf("ENV,%.2f,%.2f,%.2f\r\n", d.temperature/100.0f,
                           d.pressure/100.0f, d.humidity/1000.0f);
    }
}
}

```

3.2 UART Command Handler `console.c`

```

static char rx[128];
static uint8_t idx;
void console_process_rx(void)
{
    uint8_t c;
    while (HAL_UART_Receive(&huart2, &c, 1, 0) == HAL_OK)
    {
        if (c == '\r' || c == '\n')
        {
            rx[idx] = 0;    // NUL-terminate
            parse_cmd(rx);
            idx = 0;
        }
        else if (idx < sizeof(rx)-1)
            rx[idx++] = c;
    }
}

static void parse_cmd(char *cmd)
{
    if (!strcmp(cmd, "READ"))
    {
        // Sensor prints already handled in main loop
    }
    else if (!strncmp(cmd, "SEND,", 5))
    {
        sx126x_send(&radio, (uint8_t*)&cmd[5], strlen(&cmd[5]));
        console_puts("OK\r\n");
    }
    else if (!strcmp(cmd, "SLEEP"))
    {
        sx126x_set_standby(&radio);
        console_puts("OK\r\n");
    }
    else

```

```

    console_puts("ERR\r\n");
}

```

3.3 `sx126x_hal.c` (core SPI helper)

```

sx126x_hal_status_t sx126x_hal_write(const uint8_t *buf, uint16_t len)
{
    HAL_GPIO_WritePin(SX1262_NSS_GPIO_Port, SX1262_NSS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, (uint8_t *)buf, len, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(SX1262_NSS_GPIO_Port, SX1262_NSS_Pin, GPIO_PIN_SET);
    return SX126X_HAL_OK;
}

```

4 Build & Flash

1. **Project** > **Build Project** – resolve any `#include` errors.
2. Connect the board via the **CN1** USB and leave **BOOT0** = 0.
3. Click the *bug* icon → STM32 GDB Debugging, *Resume*.
4. Open the *SWV ITM Console* (optional) to watch `printf`.

5 Serial Test with PuTTY

1. Find *ST-Link Virtual COM Port* (e.g. COM7) in Device Manager (Windows) or `/dev/ttyACM0` (Linux).
2. Launch **PuTTY** → Serial, Port *COM7*, Speed **115200** → *Open*.
3. You should see:

```
NUCLE0-G071RB IoT ready>
```

4. Type commands + **Enter**:
5. `READ` → board prints `ENV,25.14,1022.35,45.50` every second.
6. `SEND,HelloLoRa` → responds `OK`; verify packet with a second LoRa node.
7. `SLEEP` → radio goes to standby (<2 mA).

6 Command Reference

Command	Purpose	Example	Typical Response
<code>READ</code>	Force immediate sensor sample	<code>READ</code>	<code>ENV,T,P,H</code> line
<code>SEND,<text></code>	Transmit ASCII payload over LoRa	<code>SEND,Hi</code>	<code>OK</code>

Command	Purpose	Example	Typical Response
<code>SLEEP</code>	Standby the radio	<code>SLEEP</code>	<code>OK</code>

7 Extending the Prototype

- **FreeRTOS:** enable in `.ioc`, create two tasks (sensor + radio) for deterministic latency.
- **BSEC Library:** add Bosch closed-source lib for IAQ & CO₂-eq.
- **Downlink Reception:** handle `DI01` rising IRQ, call `sx126x_get_rx_buffer_status()`.
- **AES Encryption:** wrap payload with LoRaWAN or RadioLib AES128.

Appendix A CubeMX Snapshot

```
[I2C1]
Mode=I2C
Timing=0x2000090E

[SPI1]
Mode=Full_Duplex_Master
BaudRatePrescaler=8

[USART2]
BaudRate=115200
WordLength=8
Parity=None
StopBits=1
```

Appendix B Common Pitfalls

Symptom	Likely Cause	Fix
No UART output	Wrong PuTTY line endings	Enable <i>Implicit CR in every LF</i>
<code>ENV</code> values 0	BME680 NACK	Swap 0x76/0x77 addr jumper
LoRa <code>BUSY</code> stuck high	BUSY pin not wired	Check PA1 continuity

Appendix C Helpful Links

- ST Application Note **AN2606** – UART bootloader commands
- Semtech **SX126x Datasheet** (Rev 2.1 2023-02)

Appendix D Full Source Listings

The condensed snippets above are sufficient for most users. For a ready-to-import set of `.c/.h` files (CubeIDE 1.15 workspace) see the accompanying ZIP or clone `git@github.com:your-org/iot-nucleo-g071rb-demo.git`.

End of Guide